

Contents

RDP+ v1.5.1 — Code Documentation	1
1. What the program does	1
2. Files the program depends on at runtime	1
3. Program flow	2
4. Data structures	2
5. Notable implementation details	3
6. Issues found	4
7. Build	6
8. Summary of recommendations	6

RDP+ v1.5.1 — Code Documentation

File analyzed: RDP_Plus.cpp **Author:** Glenn Madine **Language / platform:** C++17, native Win32 API (no MFC/WTl), converted from an earlier Python implementation **Dependencies:** Windows SDK, comctl32, shell32, shlwapi, user32, gdi32, and the single-header `nlohmann::json` library (`json.hpp`)

1. What the program does

RDP+ is a small Windows GUI launcher. It reads a list of remote hosts from a `connections.json` file that sits next to the executable, displays them in a sortable three-column list (Host, Type, Description), and — when the user double-clicks or presses Enter on a row — launches the appropriate remote-access tool for that host's type (RDP, SSH, a web URL, etc.). The actual command line to run for each type is itself data-driven, loaded from a second file, `actionDefinitions.json`.

In short: it's a configurable "jump list" for RDP/SSH/web/etc. connections, aimed at admins who manage many hosts and don't want to remember `mstsc.exe` arguments for each one.

2. Files the program depends on at runtime

Both files are expected in the same directory as `RDP_Plus.exe`:

File	Purpose	Loaded by
<code>connections.json</code>	The list of hosts shown in the UI	<code>LoadConnections()</code>
<code>actionDefinitions.json</code>	Maps a type string (e.g. "RDP") to the command/args used to launch it	<code>loadActions()</code>

2.1 `actionDefinitions.json` schema

A top-level JSON **array** of objects, each deserialized into an `Action`:

```
[
  { "action": "RDP", "args": " /v:", "command":
    ↪ "C:\\Windows\\System32\\MSTSC.exe" },
```

```

{ "action": "SSH", "args": " ", "command": "C:\\path\\to\\putty.exe" },
{ "action": "WEB", "args": "", "command": "https://" }
]

```

- **action** — matched against the device’s type field (uppercased) to find the right launcher.
- **command** — either an executable path, or a URL scheme prefix (e.g. `https://`) recognized specially (see §5.4).
- **args** — a string prepended to the host before it’s passed to the target program; note the code comment on line 81 calls out that a leading space is intentional (e.g. " /v:" produces `mstsc.exe /v:hostname`).

2.2 connections.json schema

`LoadConnections()` (lines 256–303) is deliberately permissive about the top-level shape. It accepts:

- A flat array of device objects: `[ {"host": "...", "type": "...", "description": "..."}, ... ]`, or
- An object whose values are either a single device object or an array of device objects (i.e., hosts grouped under arbitrary keys, such as by site or category) — the keys themselves are read but discarded.

Each device object may supply `host`, `type`, and `description` as strings; missing fields are simply left as empty wide strings (no validation or error is raised for a missing field).

3. Program flow

1. **wWinMain** (lines 690–726) initializes common controls (`ICC_LISTVIEW_CLASSES`), registers the `RDPPlusClass` window class, creates the main window (title `RDP+ v1.5.1 - Select a connection`), and runs the standard `GetMessage/DispatchMessage` loop.
2. **WM_CREATE** (lines 572–626) builds a `SysListView32` control in report mode with three columns (`Host / Type / Description`), loads `connections.json` into the global `g_devices`, and populates the list view. It also creates the two bottom buttons (“Launch RDP connection not on this list” and “Exit”).
3. **WM_SIZE** (lines 628–652) resizes the list view to fill the client area above a fixed-height button row, and centers the two buttons in that row.
4. **WM_NOTIFY** (lines 664–678) handles list view events:
 - Double-click or Enter (`NM_DBLCLK / NM_RETURN`) on a selected row calls `OnItemActivated`, which launches the connection.
 - Clicking a column header (`LVN_COLUMNCLICK`) calls `SortDevices`, repopulates the list, and updates the sort-arrow glyph on the header via `UpdateHeaderSortArrow`.
5. **WM_COMMAND** (lines 654–662) handles the two buttons: the RDP button just runs `mstsc.exe` with no arguments (a manual/ad-hoc connection); Exit destroys the window.
6. **OnItemActivated** (lines 555–564) uppercases the selected device’s type, looks up the matching `command/args` in the loaded `actionDefinitions.json` data (defaulting to the literal string `"Default"` if no match is found — see §6.5), and calls `LaunchSomething`.
7. **LaunchSomething** (lines 318–325) decides how to launch: if the resolved command begins with a recognized URL scheme (`https://`, `http://`, `ftp://`, `file://`, `mailto:`, `tel:`), it concatenates the host onto that prefix and opens it as a URL via `ShellExecuteW`. Otherwise it treats the command as an executable path and invokes it with `args + host` as the parameter string (e.g. `MSTSC.exe /v:myhost`).

4. Data structures

Type	Fields	Populated from	Used for
Action (line 79)	action, args, command (all <code>std::wstring</code>)	<code>actionDefinitions.json</code> via <code>from_json/to_json</code> ADL hooks (lines 88–100)	Deciding what to launch for a given device type
Device (line 217)	host, type, description (all <code>std::wstring</code>)	<code>connections.json</code> , via manual parsing in <code>LoadConnections</code>	Rows in the list view (<code>g_devices</code>)

Two independent parsing paths exist side by side: Action uses `nlohmann::json`’s ADL customization points (`from_json/to_json`) plus `j.get<std::vector<Action>>()`, while Device is parsed by hand inside a lambda (`parseDevice`) that checks `.contains()/is_string()` per field. This is not a bug, but it’s an inconsistency worth knowing about if the two are ever unified.

5. Notable implementation details

5.1 Two parallel UTF-8/UTF-16 conversion layers

The file defines the conversion helpers twice, with different signatures and slightly different behavior:

- `Utf8ToWide/WideToUtf8` (lines 49–76) — take/return by `const&`, throw `std::runtime_error` on failure, used by the Action/JSON-file-reading path.
- `ToWide` (line 226) / `WStringToString` (line 308) — a second, independently written pair used by the Device/LoadConnections path.

Both do essentially the same job. This duplication is a leftover of the Python→C++ conversion rather than an intentional design.

5.2 Two parallel “get exe directory” functions

- `getExeDirectory()` (lines 170–182) — manually finds the last `\` or `/` in the module path.
- `GetExeDir()` (lines 246–251) — uses `PathRemoveFileSpecW` from `Shlwapi`.

`getExeDirectory()` is used to locate `actionDefinitions.json`; `GetExeDir()` is used to locate `connections.json`. Functionally similar but not identical (one returns a trailing separator, the other does not — `WM_CREATE` compensates by hard-coding `L"\\connections.json"`).

5.3 Two file-reading strategies for the two JSON files

- `actionDefinitions.json` is read via `ReadFileUtf8()` (lines 105–128), which uses raw Win32 `CreateFileW/ReadFile` specifically so the path can be a `std::wstring` without relying on the non-standard MSVC `std::ifstream(const wchar_t*)` constructor (this rationale is stated in the comment at lines 102–104).
- `connections.json` is read via `LoadConnections()` using `std::ifstream` opened with a **narrow** path that was produced by converting the wide path to UTF-8 (lines 259–265). On Windows, the narrow-string overload of `std::ifstream` interprets that path using the process’s ANSI code page, not UTF-8. For any install location containing non-ASCII characters (a Unicode username in the profile path, a non-English “Program Files” folder name, etc.), this conversion can silently produce a path the ANSI-based `ifstream` cannot open — the exact class of bug `ReadFileUtf8` was written to avoid, reintroduced in the sibling function.

5.4 URL-vs-executable detection in LaunchSomething

The scheme check (`prefix.substr(0, N) == L"..."`) is safe even when `prefix` is shorter than the scheme being tested — `std::wstring::substr` clamps the count rather than throwing — so this isn't a crash risk, just worth knowing when reading the code.

5.5 Sorting

`SortDevices` (lines 492-514) does a case-insensitive (`_wcsicmp`) sort of `g_devices` by the clicked column, toggling ascending/descending on repeat clicks of the same column. `Update-HeaderSortArrow` then reflects that state as a `HDF_SORTUP/HDF_SORTDOWN` glyph on the list view header.

6. Issues found

These are concrete defects/risks identified while reading the code, roughly in order of severity.

6.1 `actionDefinitions.json` is reloaded and reparsed on every single window message

`WndProc` (lines 566-570) calls `getExeDirectory()`, builds the path, and calls `loadActions()` — which opens the file, reads all its bytes, and JSON-parses it — **unconditionally, at the top of the function, before the switch**. Because `WndProc` is invoked for every message the window receives (not just the ones explicitly handled — `WM_SETCURSOR`, `WM_NCHITTEST`, mouse-move messages, timers, etc. all pass through it), this means the app performs synchronous disk I/O and full JSON parsing dozens or hundreds of times per second during normal use (e.g., while the mouse moves over the window). This is a real, user-visible performance problem, and it gets worse if the JSON file is on a network share.

6.2 That same reload can crash the application

`loadActions()` (line 139, via `ReadFileUtf8` and `json::parse`) throws `std::runtime_error` (or an `nlohmann::json` parse exception) if the file is missing, locked, or malformed. Nothing in `WndProc` catches this. Since `WndProc` runs as a callback invoked by the Windows message dispatcher, an exception escaping it is undefined behavior in practice this generally unwinds through non-exception-aware system code and terminates the process. Combined with 6.1, this means a single transient hiccup reading `actionDefinitions.json` (file briefly locked by an antivirus scan, a sync client, etc.) can crash the whole app — not just at startup, but during ordinary mouse movement. Compare this with `LoadConnections()`, which wraps its own JSON parsing in a `try/catch` and shows a friendly `MessageBox` (lines 273-300) — the same safety net was not applied to the actions file.

6.3 `LoadConnections` may fail to open paths with non-ASCII characters

As described in §5.3, converting the wide file path to a narrow string via `CP_UTF8` and then handing it to `std::ifstream` (which decodes narrow paths using the ANSI code page) is inconsistent with the UTF-8 encoding used to produce it. On a machine where the install path contains characters outside the current ANSI code page, `connections.json` may fail to open even though the file exists and the same directory logic already resolves `actionDefinitions.json` successfully via the wide-path-safe `CreateFileW` route.

6.4 Substantial dead code: username-prompt dialog

Two full features related to prompting the user for a username are present but never called from anywhere in the program:

- UsernameDialog (lines 328-351), a static class with a DlgProc that reads a control ID of `IDOK + 10` — not a real, defined resource ID, so this class could not function correctly even if it were wired up.
- GetUsernamePopup() (lines 354-475), ~120 lines that hand-construct a DLGTEMPLATE/DLGITEMTEMPLATE byte layout in a fixed 512-byte stack buffer to show a small “SSH Login” / “Enter your username” dialog.

Neither is referenced from `WndProc`, `OnItemActivated`, or anywhere else. This looks like functionality that was designed (perhaps to let SSH connections prompt for a username before launching PuTTY/etc.) but never integrated, and is worth either wiring in or deleting — as-is it adds ~150 lines of maintenance surface, including a hand-rolled binary struct layout, for no runtime benefit. If it’s ever wired up, note the fixed `dlgMem[512]` buffer has no bounds checking against the hard-coded control content.

6.5 Unmatched type silently becomes the literal command "Default"

In `OnItemActivated` (lines 561-562), if a device’s type doesn’t match any action entry in `actionDefinitions.json`, `cmd` and `args` both fall back to the literal string `L"Default"` via `.value_or(L"Default")`. This is then handed to `LaunchSomething`, which will attempt `ShellExecuteW(open, L"Default", L"Default" + host, ...)` — i.e., it tries to execute a program literally named `Default`, which will fail with a Windows “file not found” shell error dialog. A misconfigured or mistyped type in `connections.json` therefore surfaces as a confusing generic Windows error rather than a clear “no launcher configured for type X” message.

6.6 Logger class is defined but never used

The `Logger` class (lines 188-213) — an append-only UTF-8 log writer built on `CreateFileW/WriteFile`, mirroring `ReadFileUtf8`’s approach — is fully implemented but never instantiated anywhere in the file. There is currently no logging in the shipped app; any diagnostic information (e.g., which JSON file failed to load, or why) is only available via the ad-hoc `MessageBox` calls in `LoadConnections`.

6.7 Two independent JSON/UTF-8 helper stacks (maintenance smell)

As detailed in §5.1–5.2, `Utf8ToWide/WideToUtf8/getExeDirectory()` and `ToWide/WStringToString/GetExeD` are redundant pairs solving the same problems slightly differently. This roughly doubles the surface area for encoding-related bugs (see 6.3, which arose specifically from the second, less careful path) and makes the code harder to maintain than necessary.

6.8 PopulateListView ignores the parameter it’s given

`PopulateListView(HWND hLV, const std::vector<Action>& actions)` (line 537) takes an `actions` parameter but its body only ever reads the global `g_devices`, never `actions`. It’s harmless today (every call site happens to have `actions` in scope anyway) but the signature is misleading — a reader would reasonably expect device data to come from the parameter, not an implicit global.

6.9 No sanitization of args + host before passing to ShellExecuteW

LaunchSomething concatenates args and host directly into the parameter string passed to ShellExecuteW (line 323). Both args (from actionDefinitions.json) and host (from connections.json) are local configuration files, so this isn't a remote attack surface by itself, but it's worth flagging for a deployment where these files might live on a shared/writable network location: anyone who can edit either JSON file controls the exact command line every user of the tool executes on double-click, with no escaping or validation of host names (e.g., a host string containing a stray " or & could alter how the target program parses its arguments).

7. Build

The header comment (lines 7-8) documents the exact build invocation:

```
CL /EHsc /W3 /O2 /GL /DUNICODE /D_UNICODE /DNDEBUG /std:c++17 RDP_Plus.cpp RDP_Plus.res
↪ /Fe:RDP_Plus.exe /link /SUBSYSTEM:WINDOWS comctl32.lib shell32.lib shlwapi.lib
↪ user32.lib gdi32.lib
```

This is a release-mode, whole-program-optimized (/GL), Unicode build, linked as a pure GUI subsystem app (/SUBSYSTEM:WINDOWS, so no console window). It expects a compiled RDP_Plus.res resource file (referenced via resource.h, providing IDI_ICON1 for the window icon) alongside the source.

8. Summary of recommendations

Ordered to match §6:

1. Load actionDefinitions.json once (e.g., on WM_CREATE, right next to where connections.json is already loaded into g_devices) and cache it in a global/member, rather than reloading it at the top of WndProc on every message.
2. Wrap that load (wherever it ends up) in a try/catch, following the same pattern already used in LoadConnections, so a bad or missing actionDefinitions.json produces a MessageBox instead of terminating the process.
3. Route connections.json through the same CreateFileW-based reader used for actionDefinitions.json (ReadFileUtf8), instead of the ANSI-code-page-sensitive std::ifstream(narrowPath) path, and parse it with nlohmann::json::parse on the resulting UTF-8 string.
4. Either integrate GetUsernamePopup/UsernameDialog into the SSH launch path or remove them; as dead code they add risk (an unfinished, unvalidated hand-built DLGTEMPLATE) without benefit.
5. Distinguish “no action configured for this type” from a real launch attempt — e.g., show a MessageBox naming the unmatched type instead of trying to execute a program literally called Default.
6. Either wire up Logger for real diagnostics (especially useful once 1-2 are fixed, to record why a load failed) or remove it.
7. Consolidate the duplicated UTF-8/UTF-16 and exe-directory helper pairs into one implementation each, reducing the chance of the two paths drifting further (as they already have, per §6.3).
8. Have PopulateListView take the device list as an explicit parameter (or drop the unused actions parameter) so its signature matches what it actually does.
9. If these JSON files could ever live on a shared/writable location, document that trust boundary (whoever can write the JSON controls what gets executed) and consider validating/escaping host before it's concatenated into a shell command line.